

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	KOTHAKOTA RAKESH
Reg. No	192525075

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

FACULTY: Dr. Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

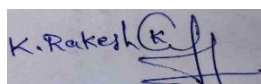
Duration: 30 minutes

Marks: 25

Code of Conduct:

I **Kothakota Rakesh/192525075** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student

Name of the Student : K.Rakesh

QUESTIONS:**Total Marks:25****1. Error Analysis in Maximum Likelihood Estimation (MLE)**

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

Answer: The conceptual error is treating the likelihood as a sum of probabilities instead of a product. For independent observations, the joint probability of the dataset is obtained by multiplying the probability of each sample under the model. Therefore, the correct likelihood must measure how probable the whole dataset is, not just the added probabilities of individual samples. for labelled classification data. If the samples are independent and identically distributed, multiplying the conditional probabilities gives the joint likelihood of all labels.

b) Explain how this mistake affects parameter estimation.

Answer: This mistake changes the objective function, so the parameters selected may not maximize the true probability of the observed data. A sum can hide poor predictions on some samples because high probabilities for other samples may compensate for them. As a result, even with large data, the estimator can become biased, inconsistent, and unable to learn the correct decision boundary.

c) Suggest the correct mathematical formulation and reasoning.

Answer: The correct formulation is $L(\theta) = \prod_i P(y_i | x_i; \theta)$ for labeled classification data. If the samples are independent and identically distributed, multiplying the conditional probabilities gives the joint likelihood of all labels given the inputs. The best parameter θ is the one that maximizes this joint likelihood. for labeled classification data. If the samples are independent and identically distributed, multiplying the conditional probabilities gives the joint likelihood of all labels

d) How would using log-likelihood fix computational issues?

Answer: Using log-likelihood converts the product into a sum: $\log L(\theta) = \sum_i \log P(y_i | x_i; \theta)$. This avoids numerical underflow caused by multiplying many small probabilities. It also makes differentiation easier, so gradient-based optimization becomes more stable and efficient. This avoids numerical underflow caused by multiplying many small probabilities. It also makes differentiation easier, so gradient-based optimization becomes more stable and efficient.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Answer: Possible implementation errors include using the wrong activation rule, updating weights even for correctly classified samples, using the wrong sign in the update equation, not updating the bias term, or stopping before enough epochs are completed. Errors in label encoding, such as using 0/1 when the update assumes -1/+1, can also prevent convergence.

b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer: A very high learning rate may cause the perceptron weights to overshoot a separating boundary, while a very low learning rate can make convergence appear extremely slow. Incorrect weight updates, such as subtracting instead of adding $\eta \cdot y \cdot x$ for a misclassified sample, move the decision boundary in the wrong direction. Since perceptron convergence depends on correct updates for misclassified points, such errors directly break convergence.

c) Identify dataset-related issues that may cause this problem.

Answer: Dataset-related problems include wrongly assuming the data is linearly separable, having mislabeled samples, duplicate samples with opposite labels, extreme outliers, or unscaled features. If the data is not actually separable due to noise or labeling mistakes, the basic perceptron convergence theorem does not apply. Poor scaling can also make one feature dominate the update direction.

d) Propose modifications to ensure convergence.

Answer : To ensure convergence, verify label encoding, update only on misclassified samples, include the bias update, and shuffle the training data across epochs. Feature scaling should be applied so all dimensions contribute fairly. If the data is not perfectly separable, use a pocket perceptron, averaged perceptron, margin perceptron, or switch to regularized logistic regression or SVM.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Answer: Increasing loss can be caused by using the wrong gradient sign, missing chain rule terms, incorrect derivative of the activation function, wrong matrix dimensions, or

averaging gradients incorrectly across the batch. Another common error is updating parameters using gradient ascent when the goal is loss minimization. A gradient check using finite differences can identify whether the computed backpropagation gradients are correct.

b) Explain the role of activation functions in gradient flow.

Answer: Activation functions control how gradients flow from later layers to earlier layers. Their derivatives multiply the upstream gradient during backpropagation. If an activation saturates, such as sigmoid near 0 or 1, its derivative becomes very small and the gradient passed backward becomes weak. Suitable activations help maintain useful gradient signals.

c) Analyze how vanishing or exploding gradients affect training.

Answer: Vanishing gradients make earlier layers learn very slowly because their updates become almost zero. Exploding gradients make updates extremely large, which can cause unstable weights and increasing loss. Both problems prevent the network from moving smoothly toward a minimum and can make training fail even when the architecture is reasonable.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer: Useful fixes include proper weight initialization such as Xavier or He initialization, input normalization, batch normalization, gradient clipping, and choosing non-saturating activations such as ReLU where appropriate. Lowering the learning rate can also prevent unstable updates. These techniques improve gradient scale and keep training numerically stable.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

Answer: Batch gradient descent is slow on large datasets because it computes the gradient using every training sample before making a single parameter update. Each iteration is computationally expensive, especially when the model has many parameters. Although the gradient direction is accurate, the number of full passes required can make convergence time high. batch normalization, gradient clipping, and choosing non-saturating activations such as ReLU where appropriate. Lowering the learning rate can also prevent unstable updates. These techniques improve gradient scale and keep training numerically stable.

b) Analyse the cause of fluctuations in SGD.

Answer: SGD shows fluctuations because each update is based on one randomly selected sample or a very small subset of data. This sample gradient is only a noisy estimate of the true gradient over the full dataset. Therefore, the loss may move up and down even when the average direction is toward a minimum.

c) Compare the error behaviour of Mini-Batch Gradient Descent.

Answer: Mini-batch gradient descent balances the behavior of batch gradient descent and SGD. It uses a small batch of samples, so the gradient estimate is less noisy than pure SGD but cheaper than full-batch gradient descent. Its error curve usually has smaller fluctuations than SGD and faster practical convergence than batch gradient descent. so the gradient estimate is less noisy than pure SGD but cheaper than full-batch gradient descent. Its error curve usually has smaller fluctuations than SGD and faster practical convergence than batch gradient descent.

d) Recommend the best approach and justify with error trade-offs.

Answer: The best approach is usually mini-batch gradient descent with a suitable batch size, learning-rate schedule, and possibly momentum or Adam. It provides a good trade-off between computational efficiency and stable error reduction. The batch size reduces variance in the gradient estimate, while the learning-rate schedule helps the model converge more smoothly near the optimum. It provides a good trade-off between computational efficiency and stable error reduction. The batch size reduces variance in the gradient estimate

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer: High dimensionality increases model error because the feature space becomes sparse and the model has many possible patterns to fit. With limited training data, the model may fit noise or random correlations instead of true relationships. This increases variance and leads to poor test performance even when training error is low.

b) Identify signs of overfitting in this scenario.

Answer: Signs of overfitting include high training accuracy but low test accuracy, very low training loss with high validation loss, and large differences between training and validation performance. The model may also be sensitive to small changes in the training data. These symptoms show that the model has memorized the training set instead of generalizing. large differences between training and validation performance. The model may also be sensitive to small changes in the training data. These symptoms show that the model has memorized the training set instead of generalizing.

c) Analyze why distance-based learning fails in high dimensions.

Answer: Distance-based learning fails in high dimensions because distances between points become less meaningful. As dimensions increase, data points tend to become similarly far apart, so nearest neighbors are no longer clearly distinguishable. Irrelevant and noisy features also dominate distance calculations, causing wrong similarity judgments. As dimensions increase, data points tend to become similarly far apart, so nearest neighbors are no longer clearly distinguishable. Irrelevant and noisy features also dominate distance calculations, causing wrong similarity judgments.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer: Dimensionality reduction techniques such as PCA, feature selection, and autoencoders reduce the number of effective input dimensions. Regularization methods such as L1, L2, dropout, and early stopping limit model complexity. These methods remove noise, reduce variance, and force the model to learn stronger general patterns instead of memorizing high-dimensional training data. As dimensions increase, data points tend to become similarly far apart, so nearest neighbours are no longer clearly distinguishable. Irrelevant and noisy features also dominate distance calculations, causing wrong similarity judgments.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand